Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

This paper presents Rings, an implemented structured peer-to-peer network, and specifies its authority boundary as a carrier-separated algebra. The motivating failure mode is authority collapse:

$$\mathsf{root}(id, data, route) \in \{\mathsf{server}, \mathsf{dns}, \mathsf{relay}, \mathsf{platform}\}.$$

Given an overlay, namespaces, protocol effects, and cryptographic sessions, the problem is to construct a total owner map while preventing non-placement carriers from changing ownership:

$$\mathsf{Model}_{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ v_0 &\in \{1, \ldots, 256\}, \\ \mathrm{StoreOwner}_N^{v_0} &: R \to N, \\ \Omega_N(X, H) &= (X, \mathrm{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\begin{aligned} \mathsf{Claim}_{core} &= \mathsf{WellDef}(\Omega_N) \wedge \mathsf{CarrierSep} \wedge \mathsf{NonInterfere} \\ &\quad \wedge \mathsf{MorphismPreserve} \wedge \mathsf{EffectTyped}, \\ \mathsf{Claim}_{ledger} &= \mathsf{EvidenceScoped}_\Gamma \wedge \mathsf{LedgerDisc}_{Rings}. \end{aligned}$$

This paper gives a bounded algebraic account of the implemented system and a ledger for which claims the paper admits:

$$\begin{aligned} \mathsf{Formalized}_{paper} &= \mathsf{Core}_{Rings}, \\ \mathsf{Admitted}_{paper} &\subseteq \mathsf{Spec} \uplus \mathsf{Artifact}_R, \\ \mathsf{Eval}_{reported} &\subset \mathsf{ReportedOnly}, \\ \mathsf{OnionImpl}_R &= \mathsf{OnionReg} \cup \mathsf{OnionCircuit}, \\ \mathsf{OnionImpl}_R &\subseteq \mathsf{Artifact}_R, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{Anon}, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{WANPerf}, \\ \mathsf{PerfClaim}_R &\in \mathsf{Obligation}. \end{aligned}$$

I. Introduction

Centralized platforms are efficient deployment mechanisms but poor authority carriers: identity, naming, storage, and traffic control collapse to service operators [1]–[3]. This paper studies Rings through the following algebraic question:

$$\begin{aligned} \mathsf{Own}(d) &= k_d, \\ \mathsf{Place}_\nu(x) &= \mathrm{owner}_N(H_\nu(x)), \\ \mathsf{Run}(\mathcal{P}_\nu) &= \tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \mathsf{RootOfAuthority} &\neq \mathsf{ApplicationServer}, \\ \mathsf{CryptoCarrier} \cap R &= \varnothing. \end{aligned}$$

Chord and CorrectChord supply the routing substrate; DID-style proofs supply self-certifying identity; WebRTC and WebAssembly supply runtime interpretation; CRDTs supply one merge algebra. None alone gives a typed law

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; last updated July 8, 2026.

for owner, carrier, and evidence scope:

$$\begin{aligned}
\mathsf{Chord/CorrectChord} &\Rightarrow \mathsf{ringLookup} \not\Rightarrow \mathsf{resourceAuthority},\\
\mathsf{DID} &\Rightarrow \mathsf{keyBoundName} \not\Rightarrow \mathsf{overlayOwner},\\
\mathsf{WebRTC} &\Rightarrow \mathsf{browserTransport} \not\Rightarrow \mathsf{protocolLaw},\\
\mathsf{CRDT} &\Rightarrow \mathsf{mergeLaw} \not\Rightarrow \mathsf{placementLaw}.
\end{aligned}$$

The contribution is a composition theorem plus a ledger discipline. The Chord identifier space is the additive cyclic group for placement and interval order; signatures, threshold schemes, encryption, and zero-knowledge protocols remain in their own fields or groups:

$$\begin{aligned}
R &= \mathbb{Z}/2^{160}\mathbb{Z},\\
\mathsf{CC} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\},\\
R \cap \mathsf{CC} &= \varnothing.
\end{aligned}$$

$$\begin{aligned}
C_1 &= \Omega_N : \mathbf{Set}/R \to \mathbf{Set}/N,\\
C_2 &= \mathsf{CarrierSep} \Rightarrow \mathsf{NonInterfere},\\
C_3 &= \mathsf{Admit}_R(c,k) \Rightarrow \mathsf{Support}_R(c,k),\\
C_4 &= \mathsf{ReportedOnly} \not\Rightarrow \mathsf{PerfClaim}_R.
\end{aligned}$$

These four claims define the paper's scientific unit.

II. Problem Statement and Design Goals

Existing decentralized systems solve orthogonal subproblems. The missing object is the commuting authority layer between identity, placement, execution, and evidence:

$$\begin{aligned}
\mathcal{N}_R &= \{(X_\nu, H_\nu)\}_\nu,\\
\mathcal{P}_R &= \{\mathcal{P}_\nu\}_\nu,\\
\mathsf{Input} &= (\Pi, \mathcal{R}_N, \mathcal{N}_R, \mathcal{P}_R, \mathcal{X}),\\
\mathsf{Need} &= \begin{cases} \mathsf{Owner} : X_\nu \to N,\\ \mathsf{Run} : \mathcal{P}_\nu \to \mathsf{Effects}, \end{cases}\\
\mathsf{Forbidden} &= \begin{cases} \mathsf{CryptoCarrier} = R,\\ \mathsf{Evidence}_\Gamma = \mathsf{UniversalClaim}. \end{cases}
\end{aligned}$$

Obligation 1 (Design target).

$$\begin{aligned}
G_{id} &= \mathsf{SelfCertID} \wedge \mathsf{DelegatedSession},\\
G_{own} &= \mathsf{StructuredOwner},\\
G_{io} &= \mathsf{BrowserNativeTransport},\\
G_{eff} &= \mathsf{TypedProtocolEffect},\\
G_{cr} &= \mathsf{CarrierCorrectCrypto},\\
G_{rep} &= \mathsf{BoundedRepairEvidence},\\
G_{bd} &= \mathsf{ExplicitClaimBoundary},\\
G_* &= G_{id} \wedge G_{own} \wedge G_{io} \wedge G_{eff}\\
&\quad \wedge G_{cr} \wedge G_{rep} \wedge G_{bd}.
\end{aligned}$$

Definition 1 (Methodology and claim ledger).

$$\begin{aligned}
\mathsf{Method} &= \mathsf{Mot} \to \mathsf{Spec} \to \mathsf{Algebra} \to \mathsf{ImplMap}\\
&\quad \to \mathsf{Evidence}_\Gamma \to \mathsf{Obligation},\\
\Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}),\\
\mathsf{ClaimKind} &= \mathsf{Spec} \cup \mathsf{Impl} \cup \mathsf{Evidence}_\Gamma \cup \mathsf{Obligation},\\
\mathsf{Ev}_\Gamma &= \mathsf{Evidence}_\Gamma,\\
\mathsf{Anchor} &= \mathsf{Label} \cup \mathsf{Path} \cup \mathsf{ArtifactId},\\
\mathsf{Scope} &= \mathsf{Finite} \cup \mathsf{AllModel} \cup \mathsf{Open},\\
\mathsf{Ledger}_R &\subseteq \mathsf{Prop}_R \times \mathsf{ClaimKind} \times \mathsf{Anchor} \times \mathsf{Scope},\\
\mathsf{row}_R(c,k,a,\sigma) &\Leftrightarrow (c,k,a,\sigma) \in \mathsf{Ledger}_R,\\
\mathsf{Prop}_R &= \{c : \exists k, a, \sigma.\mathsf{row}_R(c,k,a,\sigma)\},\\
\mathsf{Support}_R &: \mathsf{Prop}_R \times \mathsf{ClaimKind} \to \{\top, \bot\},\\
\mathsf{Support}_R(c,k) &\Leftrightarrow \exists a, \sigma.\mathsf{row}_R(c,k,a,\sigma),\\
\mathsf{Admit}(c) &\Leftrightarrow \exists k \in \mathsf{ClaimKind}.\ \mathsf{Admit}_R(c,k).
\end{aligned}$$

Definition 2 (Global symbols and non-claims).

$$\begin{aligned}
\mathsf{NS} &= \{D, V, M, H, U, Q, W\}, \qquad VID = V,\\
\mathsf{Time} &= \mathbb{T}, \quad \mathsf{Ctx} = B^*, \quad \mathsf{Cap} \subseteq A^*,\\
\mathsf{Domain} &= \mathsf{WitnessDomain}, \quad \mathsf{Order} = \{<, =, >, \bot\},\\
\mathsf{Prov} &= (\mathsf{src}, \mathsf{rev}, \mathsf{cmd}, \mathsf{seed}, \mathsf{host}),\\
\mathsf{Limit} &= \mathsf{NonClaim}_{Rings} \cup \mathsf{Obligation}.
\end{aligned}$$

$$\begin{aligned}
\mathsf{NonClaim}_{Rings} = {} & \{\mathsf{Novel(DHT)}, \mathsf{Novel(DID)}\}\\
& \cup \{\mathsf{Novel(WebRTC)}\}\\
& \cup \{\mathsf{Novel(CRDT)}, \mathsf{Novel(Crypto)}\}\\
& \cup \{\mathsf{ByzLive}, \mathsf{Consensus}, \mathsf{WANPerf}\}\\
& \cup \{\mathsf{Anon}, \mathsf{Unlinkability}, \mathsf{OnionPerf}\}\\
& \cup \{\mathsf{FullVNodeRouting}, \mathsf{VNodePerf}\}.
\end{aligned}$$

Claim family	Ledger class	Anchor and scope
Ω_N, PMor	Spec	Definitions and Theorem 1
Chord state	Spec $\cup$ Ev$_\Gamma$	Algorithms 1–3; finite checks
DID/session	Impl	signed session and context predicates
Storage VNs	Impl $\cup$ Ev$_\Gamma$	affine keys, virtual owners, storage tests
E2E frames	Impl $\cup$ Ev$_\Gamma$	signed frame and receiver policy
Onion circuits	Impl $\cup$ Ev$_\Gamma$	exit registry, route builder, layered AEAD tests
Chunking	Impl $\cup$ Ev$_\Gamma$	Algorithm 6; bounded reassembly
Deferred APIs	Spec $\cup$ Obligation	mailbox, hidden descriptor, secret sharing
Wit/rank/sec.	Spec $\cup$ Obligation	policy predicates, no global security theorem
Benchmark rows	$\notin$ ClaimKind	baseline/runtime coordinates, not admitted eval

Table I
Claim-ledger instantiation. Rows inside ClaimKind may be admitted only through an explicit ledger row (c,k,a,σ); ReportedOnly rows are coordinates, not claim evidence.

$$\begin{aligned}
\ell_{own} &= (\mathsf{OwnerLaw}, \mathsf{Spec}, \mathrm{Thm}\,1, \mathsf{AllModel}),\\
\ell_{mor} &= (\mathsf{MorphLaw}, \mathsf{Spec}, \mathrm{Thm}\,1, \mathsf{AllModel}),\\
\ell_{eff} &= (\mathsf{EffectLaw}, \mathsf{Spec}, \mathrm{Lem}\,3, \mathsf{AllModel}),\\
\ell_{car} &= (\mathsf{CarrierLaw}, \mathsf{Spec}, \mathrm{Lem}\,4, \mathsf{AllModel}),\\
\mathsf{Ledger}_{core} &= \{\ell_{own}, \ell_{mor}, \ell_{eff}, \ell_{car}\},\\
\mathsf{Ledger}_R &\supseteq \mathsf{Ledger}_{core}.
\end{aligned}$$

Definition 3 (Contribution boundary).

$$\begin{aligned}
\mathsf{Reuse}_{Rings} = {} & \{\mathsf{Chord}, \mathsf{CorrectChord}, \mathsf{DID}, \mathsf{WebRTC},\\
& \mathsf{CRDT}, \mathsf{OnionRouting}\},\\
\mathsf{New}_{Rings} = {} & (\Omega_N, \mathsf{PMor}, \mathsf{CarrierSep}, \mathsf{LedgerDisc}_{Rings}).
\end{aligned}$$

$$\begin{aligned}
\mathsf{Contribution}_{paper} &= (\mathsf{Reuse}_{Rings}, \mathsf{New}_{Rings}),\\
\mathsf{TheoremTarget}_{paper} &= \mathsf{Core}_{Rings},\\
\mathsf{LedgerTarget}_{paper} &= \mathsf{LedgerDisc}_{Rings}.
\end{aligned}$$

The formal theorem is Theorem 1; $\mathsf{LedgerDisc}_{Rings}$ is an admission discipline, not a semantic entailment from Reuse_{Rings} and not a performance theorem.

III. Algebraic System Model

Assumption 1 (System and adversary scope).

$$\begin{aligned}
\kappa &\in \mathbb{N},\\
\mathsf{fault}_{overlay} &= \mathsf{fail\text{-}stop},\\
\mathsf{Adv}_{net} &= \left\{\begin{array}{l}\mathsf{delay},\mathsf{drop},\mathsf{dup},\\ \mathsf{reorder},\mathsf{replay}\end{array}\right\},\\
\mathsf{Adv}_{id} &= \left\{\begin{array}{l}\mathsf{manyDID},\mathsf{stolenSession},\\ \mathsf{staleProof}\end{array}\right\},\\
\Pr[\mathsf{Forge}_{\Pi}(A,\kappa)] &\le \epsilon_{\Pi}(\kappa),\\
\Pr[\mathsf{Preimage}_{H}(A,\kappa)] &\le \epsilon_{H}(\kappa),\\
\epsilon_{\Pi},\epsilon_{H} &\in \mathsf{negl},\\
\mathsf{consensus} &\notin \mathsf{Overlay},\\
\mathsf{order} &= \mathsf{SidecarWitness},\\
\mathsf{crypto_ops} &\cap \mathsf{route_ops} = \varnothing.
\end{aligned}$$

Thus CorrectChord is the safety root only for crash-scoped overlay maintenance. Cryptographic claims are computational assumptions; compromised keys and Byzantine routing are adversarial cases. Eclipse resistance, Sybil resistance, anonymity, and denial-of-service resistance must be discharged by admission, session typing, ranking, relay, quota, and measurement predicates at their own layers.

Definition 4 (Identifier carrier).

$$\begin{aligned}
M &= 2^{160},\\
R &= \mathbb{Z}/M\mathbb{Z},\\
\delta_a(x) &= (x-a) \bmod M,\\
(a,b]_R &= \{x \in R : 0 < \delta_a(x) \le \delta_a(b)\},\\
(a,b)_R &= \{x \in R : 0 < \delta_a(x) < \delta_a(b)\},\\
\mathsf{RouteOps}_R &= \{0,+,-,\delta_a,(_,_]_R,(_,_)_R\},\\
\mathsf{Mult}_R &\notin \mathsf{RouteOps}_R.
\end{aligned}$$

Definition 5 (Live topology).

$$\begin{aligned}
N &\subset R, \quad 1 \le |N| < \infty,\\
\mathrm{owner}_N(k) &= \arg\min_{u\in N}\delta_k(u),\\
\mathrm{cw}_N(n) &= \mathrm{sort}_{\delta_n}(N\setminus\{n\})\cdot\langle n\rangle,\\
\mathrm{succ}_N(n) &= \mathrm{head}(\mathrm{cw}_N(n)),\\
\mathrm{pred}_N(n) &= \begin{cases}\bot, & |N|=1,\\ \arg\min_{p\in N\setminus\{n\}}\delta_p(n), & |N|>1,\end{cases}\\
\mathrm{Succ}^q_N(n) &= \mathrm{take}_{\min(q,|N|)}(\mathrm{cw}_N(n)).
\end{aligned}$$

The list $\mathrm{cw}_N(n)$ contains strictly clockwise live nodes and uses n only as the wrap-around sentinel. Hence a singleton ring has $\mathrm{Succ}^q_N(n) = \langle n\rangle$ and $\mathrm{pred}_N(n) = \bot$.

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned}
r &\in \mathbb{N}_{>0},\\
\mathsf{Steady}(N,r) &\Rightarrow |N| \ge r+1,\\
\mathsf{Init}(N,r) &\Leftrightarrow 1 \le |N| \le r,\\
|\,\mathrm{Succ}^r_N(n)| &= \min(r,|N|),\\
\mathsf{Maintain} &= \{\mathsf{join},\mathsf{rectify}\}\\
&\quad \cup \{\mathsf{stabilize}\}_{CorrectChord}.
\end{aligned}$$

CorrectChord claims in this paper are stable-ring claims under $\mathsf{Steady}(N,r)$; singleton and small-ring behavior is an explicit initialization edge case, not evidence for r distinct live successors.

Definition 6 (Affine replica set and storage VNODE ownership). Write $S^v_N = \mathrm{StoreOwner}^v_N$ and $Q^{\rho,v}_N = \mathrm{RepOwner}^{\rho,v}_N$.

$$\begin{aligned}
1 \le \rho &\le M,\\
v_0 &\in \{1,\dots,256\},\\
\mathrm{repkey}^{\rho}_i(k) &= (k + \lfloor M\cdot i/\rho\rfloor) \bmod M, \quad 0 \le i < \rho,\\
\mathrm{RepKey}^{\rho}(k) &= \langle \mathrm{repkey}^{\rho}_i(k) : 0 \le i < \rho\rangle,\\
z^v_{n,i} &= H(\texttt{rings:vnode}, net, n, i),\\
\mathrm{vnode}^v_{net}(n,i) &= z^v_{n,i},\\
V^v_N &= \{(n, z^v_{n,i}) : n \in N, 0 \le i < v\},\\
S^0_N(k) &= \mathrm{owner}_N(k),\\
y^v_k &= \arg\min_{(n,z)\in V^v_N}\delta_k(z),\\
S^v_N(k) &= \pi_1 y^v_k \quad (v>0),\\
r_i &= \mathrm{repkey}^{\rho}_i(k),\\
q_i &= S^v_N(r_i),\\
Q^{\rho,v}_N(k) &= \langle q_i : 0 \le i < \rho\rangle.
\end{aligned}$$

$$\begin{aligned}
P_k &= \mathrm{set}(\mathrm{RepKey}^{\rho}(k)),\\
O_k &= \mathrm{set}(Q^{\rho,v_0}_N(k)),\\
|P_k| &= \rho,\\
1 \le |O_k| &\le \min(\rho,|N|),\\
\mathsf{collapse}_{N,\rho,v_0}(k) &\Leftrightarrow |O_k| < \min(\rho,|N|).
\end{aligned}$$

Replica keys specify intended placement cardinality; storage VNODE ownership maps each placement key to a physical owner through derived virtual positions. The paper's default storage mode has $v_0 > 0$; $v = 0$ is the legacy off-switch. Virtual positions are not signing identities and do not own transport sessions. CorrectChord successor lists remain the topology safety root.

Definition 7 (Rings overlay model).

$$\begin{aligned}
\mathcal{R}_N &= (R, N, \mathrm{owner}_N, \mathrm{pred}_N,\\
&\quad \mathrm{Succ}^r_N, \mathrm{StoreOwner}^{v_0}_N,\\
&\quad \mathrm{RepOwner}^{\rho,v_0}_N, F, E).
\end{aligned}$$

$$\begin{aligned}
F &\subseteq N \times \mathbb{N} \times N,\\
E &= \textstyle\prod_{\nu\in\mathsf{NS}} E_\nu,\\
\mathsf{SafetyRoot}(\mathcal{R}_N) &= (\mathrm{pred}_N, \mathrm{Succ}^r_N),\\
\mathsf{PerfWitness}(\mathcal{R}_N) &= F.
\end{aligned}$$

Proposition 1 (Rotation equivariance).

$$\begin{gathered}
t \in R, \quad N + t = \{n + t : n \in N\} \Rightarrow\\
\mathrm{owner}_{N+t}(k+t) = \mathrm{owner}_N(k) + t.
\end{gathered}$$

Proof.

$$\begin{aligned}
\forall n \in N.\quad \delta_{k+t}(n+t) &= ((n+t)-(k+t)) \bmod M\\
&= \delta_k(n).
\end{aligned}$$

□

Constraint 1 (Carrier separation). Write $P_o = \mathsf{PlacementOps}$, $C_o = \mathsf{CryptoOps}$, and $R_o = \mathsf{RunOps}$.

$$\begin{aligned}
\mathsf{PlacementOps} &= \{0, +, -, \delta_a, (a,b]_R, (a,b)_R\} \\
&\quad \cup \{\mathrm{owner}_N\} \\
&\quad \cup \{\mathrm{RepKey}^{\rho}\} \\
&\quad \cup \{\mathsf{vnode}, \mathsf{StoreOwner}, \mathsf{RepOwner}\}, \\
\mathsf{CryptoOps} &= \{\times, ^{-1}, \cdot_{scalar}, \mathsf{interp}\} \\
&\quad \cup \{\mathsf{pair}, \mathsf{subgroup}\}, \\
\mathsf{TransportOps} &= \{\mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv}\}, \\
\mathsf{EffectOps} &= \{\mathsf{emit}, \mathsf{defer}, \mathsf{store}\} \\
&\quad \cup \{\mathsf{load}, \mathsf{schedule}\}, \\
\mathsf{RunOps} &= \mathsf{TransportOps} \cup \mathsf{EffectOps}, \\
\mathsf{Ops} &= \mathsf{PlacementOps} \uplus \mathsf{CryptoOps} \\
&\quad \uplus \mathsf{RunOps}, \\
P_o \cap C_o &= \varnothing, \\
P_o \cap R_o &= \varnothing, \\
\mathsf{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}, \\
\pi_R &: \mathsf{Ops} \to \mathrm{End}(R), \\
a \in P_o &\Rightarrow \pi_R(a) \in \mathrm{End}(R), \\
a \in C_o \cup R_o &\Rightarrow \begin{cases} \pi_R(a) = \mathrm{id}_R, \\ a \notin P_o. \end{cases}
\end{aligned}$$

Thus a concrete operation can affect placement only through a member of $\mathsf{PlacementOps}$. Transport, effect, and cryptographic operations may alter runtime state, payloads, or cryptographic carriers, but their induced endomorphism on R is the identity.

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \to R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	group operation plus byte-to-point adapter
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table II
Carrier boundaries in the Rings model

Table II is the carrier-level summary used by the separation constraint; it is not an additional security or performance claim.

IV. Architecture As Interfaces

Definition 8 (Layer tuple).

$$\mathcal{A} = (I, T, O, X, P, L)$$

I	$(DID, \Pi, \mathrm{Session})$
T	$(C, A_T, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$
O	$\mathcal{R}_N$
X	$(S_X, A_X, \mathsf{cap}, I_X)$
P	$\{\mathcal{P}_\nu\}_{\nu \in \mathsf{NS}}$
L	$\mathsf{App} \circ P$

The interface tuple $\mathcal{A}$ separates identity, transport, overlay, extension runtime, protocol, and application layers. Only the overlay owns $\mathcal{R}_N$; the other layers discharge typed obligations through I, T, X, P.

Definition 9 (Identity proof).

$$\Pi = (K, \Sigma, V, \mathrm{did}, \mathsf{enc}, \mathsf{ttl})$$

$$\begin{aligned}
V &: K \times \Sigma \times B^* \to \{\top, \bot\}, \\
\mathrm{did} &: K \to X_D, \\
\mathsf{enc} &: \Sigma \to B^*, \\
\mathsf{ttl} &: \Sigma \to \mathsf{Time}.
\end{aligned}$$

Invariant 1 (Typed session authority).

$$sid = (d, s, e, ctx, cap), \qquad d \in X_D, \quad s \in K, \quad now < e.$$

$$\begin{aligned}
\mathrm{Session}(d, s, e, ctx, cap) \Leftrightarrow V(k_d, \sigma_{d \to s}, d \| s \| e \| ctx \| cap) = \top \\
\wedge\ \mathrm{KeyOf}(d) = k_d \wedge \mathrm{did}(k_d) = d.
\end{aligned}$$

$$\begin{aligned}
\mathsf{HopCheck}(sid) &\equiv \mathrm{Session}(d, s, e, ctx, cap), \\
\mathsf{BoundCtx}(sid) &= (sid, \nu, path, dst), \\
\mathsf{FrameCtx}(f) &= (sid, \nu, path, dst), \\
\mathrm{Accept}_\Delta(f) &\Rightarrow \mathrm{SessionValid}(sid) \\
&\quad \wedge \mathsf{FrameCtx}(f) = \mathsf{BoundCtx}(sid).
\end{aligned}$$

Definition 10 (Transport interface).

$$T = (C, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$$

$$A_T = \{\mathsf{Offer}, \mathsf{Answer}, \mathsf{Ice}, \mathsf{Open}, \mathsf{Send}, \mathsf{Recv}, \mathsf{Close}\}.$$

$$I_T : A_T^* \to \mathsf{IO}_{WebRTC} \cup \mathsf{IO}_{Native}.$$

WebRTC signaling is a runtime adapter rather than a new overlay primitive. The Rings transport layer may batch offer, answer, and ICE material into fewer protocol actions, but correctness remains delegated to the WebRTC/ICE state machines; evaluation therefore reports connection setup separately from overlay lookup.

Definition 11 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \mathsf{cap}, I_X)$$

$$\begin{aligned}
\mathsf{cap} &\subseteq A_X, \\
I_X &: A_X^* \to \mathsf{IO}, \\
I_X(\epsilon) &= \mathsf{Noop}, \\
I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta).
\end{aligned}$$

WebAssembly is modeled only through I_X and cap: it interprets effect lists under runtime capabilities and does not change the overlay object.

Obligation 2 (Runtime confinement).

$$\begin{aligned}
\tau &: S \times I \to \mathcal{E} + (S \times O \times A_X^*), \\
\mathsf{emit}(\tau(s, i)) &\subseteq \mathsf{cap}, \\
a \notin \mathsf{cap} &\Rightarrow I_X(a) = \mathsf{Reject}(a).
\end{aligned}$$

V. Category and Effects

Definition 12 (Namespace object).

$$\begin{aligned} \nu &\in \mathsf{NS}, \\ H_\nu &: X_\nu \to R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 13 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \to \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \mathrm{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \mathrm{owner}_N \circ H_\nu : X_\nu \to N.$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \to X_\mu, \qquad H_\mu \circ f = H_\nu.$$

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\mathrm{owner}_N \circ H_\mu \circ f = \mathrm{owner}_N \circ H_\nu.$$

□

Definition 14 (Writer effect monad).

$$\begin{aligned} A^* &= \mathsf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg\!= f &= \mathsf{let}\ f(x) = (y, \beta)\ \mathsf{in}\ (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^\star \circ g^\star) \circ f^\star = h^\star \circ (g^\star \circ f^\star).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

□

Definition 15 (Typed failure stack).

$$\mathsf{T}_A^{\mathcal{E}}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \mathsf{RecoverableReport} &\in A. \end{aligned}$$

Definition 16 (Protocol object).

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu)$$

$$\tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Definition 17 (Protocol morphism).

$$\Sigma : \mathcal{P}_\nu \to \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A, \sigma_A^*)$$

$$\begin{aligned} \sigma_X &: X_\nu \to X_\mu, \qquad \sigma_S : S_\nu \to S_\mu, \\ \sigma_I &: I_\nu \to I_\mu, \qquad \sigma_O : O_\nu \to O_\mu, \\ \sigma_\mathcal{E} &: \mathcal{E}_\nu \to \mathcal{E}_\mu, \qquad \sigma_A : A_\nu \to A_\mu, \\ \sigma_A^* &: A_\nu^* \to A_\mu^*, \\ \sigma_A^*(\epsilon) &= \epsilon, \\ \sigma_A^*(a \cdot \alpha) &= \sigma_A(a) \cdot \sigma_A^*(\alpha), \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu. \end{aligned}$$

$$\begin{aligned} \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(e) &= \sigma_\mathcal{E}(e), \\ \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(s, o, \alpha) &= (\sigma_S(s), \sigma_O(o), \sigma_A^*(\alpha)), \\ \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}} \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

The predicate $\mathsf{PMor}_{\nu\mu}(\Sigma)$ is the conjunction of these typing and preservation equations.

Lemma 1 (Owner totality). If $1 \le |N| < \infty$ and $H_\nu : X_\nu \to R$, then

$$P_\nu^N = \mathrm{owner}_N \circ H_\nu : X_\nu \to N$$

is total.

Proof.

$$\forall k \in R. \quad \mathrm{owner}_N(k) = \arg\min_{u \in N} \delta_k(u) \in N.$$

Finite nonempty N gives existence; $\delta_k : R \to \mathbb{N}$ gives a total order key. □

Lemma 2 (Morphism preservation).

$$\mathsf{PMor}_{\nu\mu}(\Sigma_{\nu\mu}) \Rightarrow P_\mu^N \circ \sigma_X = P_\nu^N.$$

Proof.

$$P_\mu^N \circ \sigma_X = \mathrm{owner}_N \circ H_\mu \circ \sigma_X = \mathrm{owner}_N \circ H_\nu = P_\nu^N. \quad \square$$

Lemma 3 (Effect confinement).

$$\forall \nu \in \mathsf{NS}. \forall s, i. \quad \tau_\nu(s, i) \in \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Moreover, if $\mathsf{emit}(\tau_\nu(s, i)) \subseteq \mathsf{cap}$, then

$$I_X(\pi_A(\tau_\nu(s, i))) \in \mathsf{IO} \quad \wedge \quad a \notin \mathsf{cap} \Rightarrow I_X(a) = \mathsf{Reject}(a).$$

Proof. The first formula is the type of τ_ν. The second is exactly the runtime confinement obligation for the interpreter I_X. □

Lemma 4 (Carrier non-interference).

$$\begin{aligned} a &\in \mathsf{CryptoOps} \cup \mathsf{RunOps} \\ &\Rightarrow \forall (X, H) \in \mathbf{Set}/R. \\ &\quad \begin{cases} \Omega_N(X, \pi_R(a) \circ H) = \Omega_N(X, H), \\ \Omega_N(X, H) = (X, \mathrm{owner}_N \circ H). \end{cases} \end{aligned}$$

Proof. By Constraint 1, such a has $\pi_R(a) = \mathrm{id}_R$. Hence $\mathrm{owner}_N \circ \pi_R(a) \circ H = \mathrm{owner}_N \circ H$. □

Definition 18 (Correct base predicate).

$$\mathsf{CorrectBase}(N, S) \Rightarrow \begin{cases} 1 \le |N| < \infty, \\ \forall k \in R. \mathrm{owner}_N(k) \in N, \\ \mathsf{CorrectChord}(N, S). \end{cases}$$

The theorem below uses only the displayed total-owner consequence. The stronger steady-state lookup obligation is stated in the Chord section as $\mathrm{Inv}_{ring}(N, S) \wedge \mathsf{FingerInv}(N, S)$.

Theorem 1 (Carrier-separated ownership theorem). Assume $\mathsf{CorrectBase}(N, S)$, carrier separation, and protocol morphisms satisfying Definition 17. Then

$$\begin{aligned}
\mathsf{OwnerLaw} &\equiv \forall \nu, x.\\
&\quad P^N_\nu(x) = \mathrm{owner}_N(H_\nu(x)) \in N,\\
\mathsf{MorphLaw} &\equiv \forall \nu, \mu, \Sigma.\\
&\quad \mathsf{PMor}_{\nu\mu}(\Sigma) \Rightarrow P^N_\mu \circ \sigma_X = P^N_\nu,\\
\mathsf{EffectLaw} &\equiv \forall \nu, s, i.\\
&\quad \tau_\nu(s,i) \in \mathsf{T}^{\mathcal{E}_\nu}_{A_\nu}(S_\nu \times O_\nu),\\
\mathsf{CarrierLaw} &\equiv \forall a \in \mathsf{CryptoOps} \cup \mathsf{RunOps}.\\
&\quad \forall (X,H) \in \mathbf{Set}/R.\\
&\quad \Omega_N(X, \pi_R(a) \circ H) = \Omega_N(X,H),\\
\mathsf{Core}_{Rings} &\equiv \mathsf{OwnerLaw} \wedge \mathsf{MorphLaw}\\
&\quad \wedge \mathsf{EffectLaw} \wedge \mathsf{CarrierLaw}.
\end{aligned}$$

Proof. OwnerLaw follows from Definition 18 and Lemma 1; MorphLaw is Lemma 2; EffectLaw is Lemma 3; CarrierLaw is Lemma 4. □

Constraint 2 (Claim ledger discipline).

$$\begin{aligned}
\mathsf{Admit}_R(c,k) &\Leftrightarrow k \in \mathsf{ClaimKind} \wedge \exists a, \sigma. \mathsf{row}_R(c,k,a,\sigma),\\
\mathsf{Admit}_R(c, \mathsf{Ev}_\Gamma) &\Rightarrow \mathsf{finite}(\Gamma) \wedge \mathsf{declared}(\Gamma),\\
\mathsf{row}_R(c, \mathsf{Spec}, a, \sigma) &\Rightarrow c \in \left\{ \begin{array}{l} \mathsf{OwnerLaw}, \mathsf{MorphLaw},\\ \mathsf{EffectLaw}, \mathsf{CarrierLaw} \end{array} \right.\\
&\quad \wedge a \in \mathsf{Anchor},\\
\mathsf{row}_R(c, \mathsf{Impl}, a, \sigma) &\Rightarrow a \in \mathsf{Manifest}_R,\\
\mathsf{row}_R(c, \mathsf{Ev}_\Gamma, a, \sigma) &\Rightarrow \mathsf{Witness}_R(c,\Gamma) \wedge a \in \mathsf{Manifest}_R,\\
\mathsf{row}_R(c, \mathsf{Obligation}, a, \sigma) &\Rightarrow c \in \mathsf{Obligation} \wedge \sigma = \mathsf{Open},\\
\mathsf{ReportedOnly} &\notin \mathsf{ClaimKind},\\
\mathsf{Reject}_R(c) &\Leftrightarrow c \in \mathsf{NonClaim}_{Rings},\\
\mathsf{Reject}_R(c) &\Rightarrow \neg \exists k.\ \mathsf{Admit}_R(c,k),\\
\mathsf{LedgerDisc}_{Rings} &= \mathsf{Admit}_R \wedge \mathsf{Reject}_R.
\end{aligned}$$

Thus non-claims are excluded by the admission judgment; they are not proved unreachable by Theorem 1.

Proposition 4 (Claim-ledger admissibility).

$$\begin{aligned}
\mathsf{Admit}_R(c,k) &\Rightarrow \exists a, \sigma. \mathsf{row}_R(c,k,a,\sigma),\\
\mathsf{row}_R(c,k,a,\sigma) &\Rightarrow \mathsf{Support}_R(c,k),\\
c \in \mathsf{NonClaim}_{Rings} &\Rightarrow \neg \mathsf{Admit}(c),\\
\mathsf{ReportedOnly} \notin \mathsf{ClaimKind} &\Rightarrow \neg \mathsf{Admit}_R(c, \mathsf{ReportedOnly}),\\
\mathsf{Admit}_R(c, \mathsf{Ev}_\Gamma) &\Rightarrow \left\{ \begin{array}{l} \mathsf{finite}(\Gamma),\\ \mathsf{declared}(\Gamma). \end{array} \right.
\end{aligned}$$

Proof. The first and third lines are Constraint 2. The second line unfolds the definition of $\mathsf{Support}_R$. The fourth line follows because admission quantifies only over $\mathsf{ClaimKind}$. □

VI. Chord Overlay

$$\begin{aligned}
\mathsf{Base} &= \mathsf{Chord},\\
\mathsf{Maintenance} &= \mathsf{CorrectChord},\\
\mathsf{Object} &= \mathcal{R}_N,\\
\mathsf{Witnesses} &= \{\text{Algorithms } 1, 2, 3\}.
\end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 19 (Local node state and Chord auxiliaries).

$$\begin{aligned}
S_n &= (p_n, L_n, F_n, E_n),\\
n &\in N \subset R,\\
p_n &\in N \cup \{\bot\}, \quad L_n \in N^{\leq r},\\
F_n &: \{0, \ldots, 159\} \rightharpoonup N.
\end{aligned}$$

The algorithms are written over identifiers $n \in N$; an implementation node record $\hat{n}$ is projected by $\mathrm{id}(\hat{n}) = n$.

$$\begin{aligned}
\mathsf{head}(\langle\rangle) &= \bot,\\
\mathsf{Normalize}_r(n, X) &= \mathrm{take}_{\min(r, |N|)}(U_n(X)),\\
U_n(X) &= \mathrm{uniq}(\mathrm{filter}_N(X) \cdot \mathrm{cw}_N(n)),\\
\mathrm{target}_i(n) &= n + 2^i \bmod M,\\
\mathrm{finger}_i(n) &= \mathrm{owner}_N(\mathrm{target}_i(n)),\\
\mathsf{FingerInv}(N,S) &\Leftrightarrow \forall n \in N. \forall i < 160.\\
&\quad \left\{ \begin{array}{l} F_n(i) \downarrow,\\ F_n(i) = \mathrm{finger}_i(n). \end{array} \right. ,\\
\mathsf{RouteSet}_S(n) &= \mathrm{rng}(F_n) \cup \mathrm{set}(L_n),\\
\mathsf{Cand}_S(n,k) &= \mathsf{RouteSet}_S(n) \cap (n,k)_R,\\
\mathsf{ClosestPreceding}_S(n,k) &= \max_{\delta_n} \mathsf{Cand}_S(n,k).
\end{aligned}$$

Here $\mathsf{ClosestPreceding}_S(n,k) = \bot$ when the displayed set is empty.

Algorithm 1 DHT Join and Rectify Transitions

```
1: function BeginJoin(n, known)
2:     pred[n] ← ⊥
3:     if known = ⊥ then
4:         succ[n] ← ⟨n⟩
5:         return Done
6:     return Remote(known, FindSuccessor(n))
7: function InstallSuccessor(n, s)
8:     if s = ⊥ then
9:         return Repair(DiscoverSuccessor(n))
10:    succ[n] ← Normalize_r(n, ⟨s⟩)
11:    return Remote(s, QuerySuccList)
12: function InstallSuccList(n, s, list)
13:    succ[n] ← Normalize_r(n, ⟨s⟩ · list)
14:    h ← head(succ[n])
15:    if h = ⊥ then
16:        return Repair(DiscoverSuccessor(n))
17:    else if h = n then
18:        return Done
19:    else
20:        a ← Remote(h, Notify(n))
21:        b ← Remote(h, SyncStorage(n))
22:        return Multi(a, b)
23: function HandleNotify(n, q)
24:    if q ≠ n ∧ (pred[n] = ⊥ ∨ q ∈ (pred[n], n)_R) then
25:        pred[n] ← q
26:    return Noop
```

Figure 1 illustrates finger targets only; lookup correctness is the interval invariant below.

Algorithm 4 fixes the notation for the biased circular order used by placement comparisons.

Algorithm 2 DHT Lookup Algorithm

```
1: function FindSuccessor(key, n)
2:     if key = n then
3:         return Found(n)
4:     s ← head(succ[n])
5:     if s = ⊥ then
6:         return Repair(QuerySuccList(n))
7:     else if s = n then
8:         return Found(n)
9:     else if key ∈ (n, s]_R then
10:        return Found(s)
11:    next ← ClosestPreceding_S(n, key)
12:    if next = ⊥ or next = n then
13:        next ← s
14:    return Remote(next, FindSuccessor(key))
```

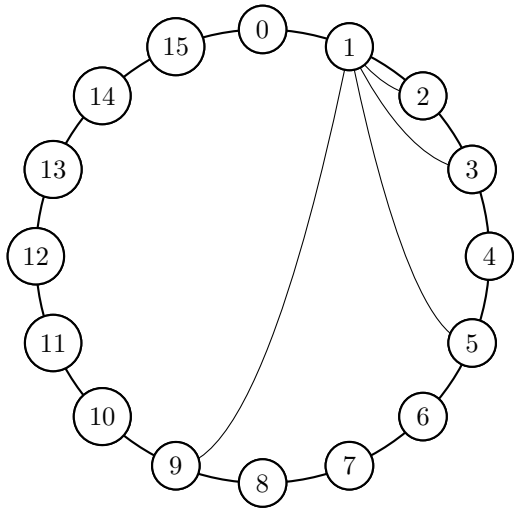

Figure 1. Chord finger targets from node 1 in a 16-node ring

Invariant 2 (Ring fixpoint).

$$\begin{aligned}\mathrm{Inv}_{ring}(N,S) \equiv \forall n \in N.\ & p_n = \mathrm{pred}_N(n) \\ & \wedge L_n = \mathrm{Succ}^r_N(n) \\ & \wedge \mathsf{head}(L_n) = \mathrm{succ}_N(n).\end{aligned}$$

Invariant 3 (Lookup owner preservation).

$$\mathsf{Eval}_S(n,k) \Downarrow o$$

be the fair closure of Algorithm 2, where $Remote(m, FindSuccessor(k))$ resumes the same lookup at node m. Then

$$\begin{aligned}&(\mathsf{Steady}(N,r) \vee |N| = 1) \wedge \mathrm{Inv}_{ring}(N,S) \\ &\Rightarrow \forall n \in N. \forall k \in R.\ \mathsf{Eval}_S(n,k) \Downarrow \mathrm{owner}_N(k).\end{aligned}$$

Proof obligation. If Algorithm 2 returns $Found(s)$ by the interval test, then $s = head(L_n) = \mathrm{succ}_N(n)$ and no live node lies in $(n,s)_R$; therefore $\mathrm{owner}_N(k) = s$. If it forwards, the chosen target is either the closest preceding finger/list entry or the first successor; in both cases it lies in $(n,k)_R$, so $\delta_{next}(k) < \delta_n(k)$. Finiteness of N gives termination. The branches $key = n$ and $s = n$ discharge the singleton and exact-owner edge cases. Repair branches are outside the steady-ring premise. □

Algorithm 3 DHT Stabilization

```
1: function BeginStabilize(n)
2:     s ← head(succ[n])
3:     if s = ⊥ then
4:         return Repair(DiscoverSuccessor(n))
5:     else if s = n then
6:         return Noop
7:     else
8:         return Remote(s, QueryTopology)
9: procedure Stabilize(n, topo)
10:    actions ← []
11:    s ← head(succ[n])
12:    if s = ⊥ then
13:        return Repair(DiscoverSuccessor(n))
14:    else if s = n then
15:        return Noop
16:    c ← topo.pred
17:    if c ≠ ⊥ and c ≠ n and c ∈ (n, s)_R then
18:        succ[n] ← Normalize_r(n, ⟨c⟩ · succ[n])
19:    succ[n] ← Normalize_r(n, succ[n] · topo.succ)
20:    h ← head(succ[n])
21:    if h ≠ ⊥ and h ≠ n then
22:        a ← Remote(h, Notify(n))
23:        b ← Remote(h, QuerySuccList)
24:        actions ← actions · [a, b]
25:    return Multi(actions)
```

VII. DID and Resource Algebra

Definition 20 (DID namespace).

$$\mathcal{D} = (X_D, H_D, \Pi, \mathrm{Session}), \qquad H_D : X_D \to R.$$

$$\begin{aligned} D_K(k) &= H_D(\mathrm{did}(k)), \\ \mathrm{owner}_N(D_K(k)) &\in N, \\ \mathrm{Session} &\subseteq X_D \times K \times \mathsf{Time} \times \mathsf{Ctx} \times \mathsf{Cap}. \end{aligned}$$

Definition 21 (VID namespace).

$$\begin{aligned} \mathcal{V} &= (X_V, H_V, \mathrm{Auth}_V), \\ H_V &: X_V \to R, \\ \mathrm{PrivateKey}(H_V(x)) &= \bot, \\ \mathrm{Auth}_V(op, x, \sigma, caps) &\in \{\top, \bot\}. \end{aligned}$$

Definition 22 (Biased order).

$$A \preceq_C B \iff \delta_C(A) \le \delta_C(B).$$

Definition 23 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \bot), \\ \mathcal{E}_v^{final} &= (E_v, \mathsf{conflict}_v, \mathsf{finalize}_v). \end{aligned}$$

$$\begin{aligned} x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \bot &= x, \\ \mathsf{finalize}_v &: \mathsf{conflict}_v(E_v) \to E_v. \end{aligned}$$

Algorithm 4 Biased Circular Comparison

```
function BiasedCompare(A, B, C)
    a ← δ_C(A)
    b ← δ_C(B)
    if a < b then
        return A <_C B
    else if a > b then
        return B <_C A
    else
        return A = B
```

$$\mathsf{case}_1 = \mathsf{CRDT}.$$

This is the CRDT join-semilattice case [6].

Invariant 4 (Placement).

$$\begin{aligned} P_{\nu,x} &= \mathrm{set}(\mathrm{RepKey}^{\rho}(H_\nu(x))),\\ K^{impl}_{\nu,x} &\subseteq R,\\ \mathrm{Inv}_{place} &\equiv \forall \nu, x.\ K^{impl}_{\nu,x} \subseteq P_{\nu,x}. \end{aligned}$$

Obligation 3 (Replica realization).

$$\begin{aligned} O_{\nu,x} &= \mathrm{set}(\mathrm{RepOwner}^{\rho,v_0}_N(H_\nu(x))),\\ L_{\nu,x} &= |O_{\nu,x}|,\\ \mathsf{FullRed}_{\nu,x} &\Leftrightarrow L_{\nu,x} = \min(\rho, |N|),\\ \mathsf{Collapse}_{\nu,x} &\Leftrightarrow L_{\nu,x} < \min(\rho, |N|),\\ \mathsf{LiveClaim}_{\nu,x}(\rho, v_0) &\Leftrightarrow \mathsf{Admit}_R(\mathsf{FullRed}_{\nu,x}, \mathsf{Evidence}_\Gamma),\\ \mathsf{LiveClaim}_{\nu,x}(\rho, v_0) &\Rightarrow \mathrm{Inv}_{place} \wedge \mathsf{FullRed}_{\nu,x},\\ \mathsf{ReplicaGap}_R &= \{(\nu, x) : \mathsf{Collapse}_{\nu,x}\},\\ \mathsf{ReplicaGap}_R \neq \varnothing &\Rightarrow \mathsf{fullReplicaRealization} \in \mathsf{Obligation}. \end{aligned}$$

When $|N| < \rho$, fewer than ρ distinct live owners is not collapse; collapse means fewer owners than the live topology can realize. For the paper default $v_0 > 0$, $O_{\nu,x}$ is the image of the storage VNODE owner map. This is a placement claim, not a full routing-VNODE or load-balance theorem.

Definition 24 (Mailbox).

$$\mathsf{Class}(\mathsf{Mailbox}) \subseteq \{\mathsf{Spec}, \mathsf{Obligation}\}.$$

$$\begin{aligned} mbox(d, e) &= H_M(\texttt{rings.mailbox} \| d \| e),\\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 25 (Hidden service descriptor).

$$\mathsf{Class}(\mathsf{HiddenDesc}) \subseteq \{\mathsf{Spec}, \mathsf{Obligation}\}.$$

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay,\\ &\quad \mathsf{Cap}, K, exp, rank),\\ \mathrm{vid}(h) &= H_H(h),\\ \mathsf{Lookup}(h) &= P^N_H(h),\\ \mathsf{Valid}(h, \sigma) &= \mathrm{Auth}_H(h, \sigma). \end{aligned}$$

Definition 26 (Subring).

$$\mathsf{Class}(\mathsf{Subring}) = \mathsf{Impl}.$$

$$\mathcal{S}_u = (u, N_u, \mathrm{owner}_{N_u}, \mathrm{Succ}^{r_u}_{N_u}, E_u)$$

$$\begin{aligned} u &= H_U(policy),\\ memberlog_u &\in \mathsf{MergeLog} \cup \mathsf{FinalState}. \end{aligned}$$

VIII. Cryptographic Carriers and Traffic

Definition 27 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q,\\ \mathsf{Scalar}(\mathbb{G}) &= \mathbb{Z}_q,\\ x &\in \mathbb{Z}_q,\\ h &= g^x,\\ \mathbb{G} &\neq R. \end{aligned}$$

Algorithm 5 ElGamal Encryption and Decryption

```
Input: Encoded group element m_G ∈ 𝔾, private key x ∈ ℤ_q, public key h = g^x, generator g
Output: Group ciphertext (c_1, c_2)
procedure Encryption
    Choose a random integer k ∈ ℤ_q
    Compute c_1 = g^k
    Compute c_2 = m_G · h^k
    return (c_1, c_2)
procedure Decryption
    Compute m_G = c_2 · (c_1)^{-x}
    return m_G
```

Algorithm 5 is the group-level carrier operation. Byte payloads must first be encoded into curve points and split into bounded frames. The direct ElGamal body is malleable when detached from its signed transport envelope. The Rings frame layer therefore binds message signatures, DID ownership of the public key, stream identifiers, sequence numbers, and final-frame checks into the security boundary rather than treating them as optional metadata.

Definition 28 (Onion exit registry and circuit).

$$\mathsf{Class}(\mathsf{OnionImpl}_R) \subseteq \{\mathsf{Impl}, \mathsf{Evidence}_\Gamma\}.$$

$$\begin{aligned} \mathsf{Topic}_{onion} &= \texttt{onion\_exits},\\ e &= (d, k_d, s, nt, net, svc, tr, \pi,\\ &\quad t_0, t_h, t_x, \sigma),\\ p &= (svc, tr, \chi),\\ \mathrm{ExitOK}(e, t) &\Leftrightarrow V(k_d, \sigma, e \setminus \sigma) = \top\\ &\quad \wedge \mathrm{did}(k_d) = d \wedge t < t_x\\ &\quad \wedge net = net_R \wedge svc \in \mathsf{Svc},\\ \mathrm{SvcOK}(e, svc) &\Leftrightarrow \mathrm{offers}(e, svc),\\ \mathrm{ProxyOK}(e, p) &\Leftrightarrow \mathrm{SvcOK}(e, svc)\\ &\quad \wedge \mathrm{transport}(e) = tr\\ &\quad \wedge \mathrm{allows}(\pi(e), \chi),\\ q &= (d, s),\\ \mathrm{RelayOK}(q, t) &\Leftrightarrow \mathrm{Online}(d, s, t)\\ &\quad \wedge \texttt{onion-relay} \in cap(d, t),\\ \rho &= (q_1, \ldots, q_{h-1}, e),\\ &\quad 1 \le h \le 8,\\ h_0 &= 3,\\ \mathrm{RouteOK}(\rho, svc, t) &\Leftrightarrow \mathrm{NoRepeat}(d(q_1), \ldots, d(e))\\ &\quad \wedge \mathrm{ExitOK}(e, t) \wedge \mathrm{SvcOK}(e, svc),\\ \mathrm{PRouteOK}(\rho, p, t) &\Leftrightarrow \mathrm{RouteOK}(\rho, svc, t)\\ &\quad \wedge \mathrm{ProxyOK}(e, p),\\ L_h &= \mathrm{AEAD}_{s(e)}(ret, svc, m),\\ L_i &= \mathrm{AEAD}_{s(q_i)}(d(q_{i+1}), cid_{i+1},\\ &\quad L_{i+1}), \quad i < h,\\ B_e &= \mathrm{AEAD}_{s_{client}}(rid, m, \sigma_e). \end{aligned}$$

Here e is the signed onion-exit descriptor, π is its signed policy, $p = (svc, tr, \chi)$ is a concrete proxy request, χ is its

target, q_i is a relay descriptor from live online-node presence, L_i is the forward layer, and B_e is a client-encrypted backward payload. RouteOK is the service-route builder; PRouteOK adds the implemented TCP/HTTPS transport and target-policy checks. Definition 28 is an implementation claim about directory, route construction, and encrypted forward/backward frames:

$$\begin{aligned} \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{Anon}, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{Unlinkability}, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{DoSResist}. \end{aligned}$$

Definition 29 (Signed encrypted frame).

$$f = (sid, seq, last, cipher, pk_s, \sigma_s).$$

$$\begin{aligned} sid &= (d, s, e, ctx, cap), \qquad d \in X_D, \quad s \in K, \\ m_f &= sid \| seq \| last \| cipher, \\ \mathrm{FrameValid}(f) \Leftrightarrow V(pk_s, \sigma_s, m_f) &= \top \\ &\wedge\ \mathrm{Session}(d, s, e, ctx, cap) \\ &\wedge\ pk_s = s \\ &\wedge\ \mathsf{FrameCtx}(f) = \mathsf{BoundCtx}(sid). \end{aligned}$$

Definition 30 (Receiver sequence policy).

$$\Delta = (next, final, closed, pending, seen_{seq}, seen_{fid}, w).$$

$$\begin{aligned} \mathsf{pending} &: \mathbb{N} \rightharpoonup \mathsf{Frame}, \\ \mathsf{NoFuture}_\Delta(seq) &\Leftrightarrow \neg\exists j \in \mathrm{dom}(pending).\ j > seq, \\ \mathsf{Dup}_\Delta(f) &\Leftrightarrow seq(f) \in seen_{seq} \\ &\quad \vee \mathrm{fid}(f) \in seen_{fid} \\ &\quad \vee pending(seq(f)) = f, \\ \mathsf{Conflict}_\Delta(f) &\Leftrightarrow seq(f) \in \mathrm{dom}(pending) \\ &\quad \wedge pending(seq(f)) \neq f, \\ \mathsf{New}_\Delta(f) &\Leftrightarrow seq(f) \notin seen_{seq} \cup \mathrm{dom}(pending) \\ &\quad \wedge \mathrm{fid}(f) \notin seen_{fid} \\ &\quad \wedge \neg closed \\ &\quad \wedge seq(f) - next \leq w \\ &\quad \wedge (final = \bot \vee seq(f) \leq final) \\ &\quad \wedge (last(f) \Rightarrow \mathsf{NoFuture}_\Delta(seq(f))), \\ \mathrm{Accept}_\Delta(f) &\Leftrightarrow \mathrm{FrameValid}(f) \\ &\quad \wedge \mathsf{New}_\Delta(f), \\ \mathrm{Reject}_\Delta(f) &\Leftarrow \mathrm{FrameValid}(f) \\ &\quad \wedge \mathsf{Conflict}_\Delta(f), \\ \mathrm{DropDup}_\Delta(f) &\Leftrightarrow \mathrm{FrameValid}(f) \\ &\quad \wedge \mathsf{Dup}_\Delta(f) \\ &\quad \wedge \mathsf{emit} = \epsilon. \end{aligned}$$

Here w is receiver policy, not a signed wire field. A byte-identical duplicate may be valid and dropped; a same-sequence different pending frame is rejected as equivocation.

Definition 31 (Secret sharing placement).

$$\mathsf{Class}(\mathsf{SecretSharing}) \subseteq \{\mathsf{Spec}, \mathsf{Obligation}\}.$$

$$\begin{aligned} P &\in \mathbb{F}_q[X], \\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\ H_S(sh_i) &\in R, \\ \mathrm{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 32 (Relay frame).

$$r = (path, dst, \nu, session, payload, report).$$

$$\begin{aligned} \mathrm{RelayValid}(r) &= \mathrm{SessionValid}(session) \\ &\quad \wedge\ dst \in R, \\ \mathrm{SplitVocabulary} &= \mathsf{MSRP}. \end{aligned}$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```
procedure E2E Chunking
    Input: data, chunk size
    Output: chunks
    chunks ← []
    total-chunks ← ceil(len(data)/chunk-size)
    for i in range(total-chunks) do
        chunk ← data[i*chunk-size : (i+1)*chunk-size]
        append chunk to chunks
    return chunks
```

Algorithm 6 gives the bounded split/reassembly shape; it does not claim MSRP interruption or interleaving semantics.

Invariant 5 (Chunk reassembly).

$$\begin{aligned} \mathrm{complete}(m) \Leftrightarrow \forall i < total(m).\ \exists! c_i \\ \wedge \sum_i |c_i| \leq B_m \\ \wedge\ now < ttl(m). \end{aligned}$$

IX. Ordering, Ranking, and Security

Definition 33 (Sidecar witness).

$$\mathsf{Class}(\mathsf{SidecarWitness}) \subseteq \{\mathsf{Spec}, \mathsf{Obligation}\}.$$

$$w = (domain, root, height, time, finality).$$

$$\begin{aligned} c &= H(msg), \\ \mathrm{Witness}(msg, w) \Leftrightarrow \mathrm{Include}(c, w.root) &= \top \\ \wedge\ \mathrm{Final}(w) &= \top. \end{aligned}$$

Definition 34 (Witness order).

$$\begin{aligned} a \prec b \Leftrightarrow domain(w_a) &= domain(w_b) \\ \wedge\ time(w_a) &< time(w_b). \end{aligned}$$

$$\chi : \mathsf{Domain} \times \mathsf{Domain} \to \mathsf{Order}.$$

Definition 35 (Ranking event).

$$\mathsf{Class}(\mathsf{Ranking}) \subseteq \{\mathsf{Spec}, \mathsf{Obligation}\}.$$

$$q = (subject, rater, behavior, evidence, epoch, weight, \sigma).$$

$$\begin{aligned} m_q &= subject \| behavior \| evidence \| epoch, \\ \mathrm{RankValid}(q) \Leftrightarrow V(k_{rater}, \sigma, m_q) &= \top. \end{aligned}$$

Definition 36 (Score aggregation).

$$score_e(s) = \text{robust}\{weight(q) \cdot value(q) : q.subject = s\}.$$

$$\text{robust} \in \{\mathsf{median}, \mathsf{trimmedMean}, \mathsf{declared}\}.$$

Attack	Predicate	Boundary claimed here
Sybil	one actor controls many DIDs	not solved by overlay; admission, ranking, quota
Eclipse	biased neighbor or successor view	not claimed under Byzantine routing; requires secure-routing policy
Replay	stale valid signature or frame	typed session, expiry, nonce/sequence set
MITM	wrong key for DID or session	DID proof, signed envelope, context binding
DoS	unbounded storage, relay, or chunks	quotas, TTL, chunk bounds, backpressure
Privacy	route or relay observation	relay policy only; no anonymity theorem

Table III
Threat predicates and claim boundaries

Table III fixes the negative boundary: listed attacks are not discharged by the overlay theorem.

Definition 37 (Security and policy boundary).

$$\begin{aligned}
\mathsf{AllowedRelay} &\subseteq \mathsf{Path},\\
\text{use}, \text{limit} &: \mathsf{User} \times \mathsf{Epoch} \to \mathbb{N},\\
\mathsf{SecEvent} &\subseteq X_D \times \mathsf{Epoch} \times \mathsf{RankEvent} \times \mathsf{RelayFrame} \times \mathsf{User},\\
\mathsf{P}_{adm} &: X_D \times \mathsf{Epoch} \to \{\top, \bot\},\\
\mathsf{P}_{rank}(q) &\Leftrightarrow \text{RankValid}(q) \wedge weight(q) \le W_{max},\\
\mathsf{P}_{relay}(r) &\Leftrightarrow \text{RelayValid}(r) \wedge \text{path}(r) \in \mathsf{AllowedRelay},\\
\mathsf{P}_{quota}(u,e) &\Leftrightarrow \text{use}(u,e) \le \text{limit}(u,e),\\
z &= (d,e,q,r,u),\\
\mathsf{Policy}_R(z) &\Leftrightarrow \mathsf{P}_{adm}(d,e) \wedge \mathsf{P}_{rank}(q) \wedge \mathsf{P}_{relay}(r) \wedge \mathsf{P}_{quota}(u,e).
\end{aligned}$$

$$\begin{aligned}
\mathsf{Base} &\not\vdash \mathsf{SybilResist} \wedge \mathsf{EclipseResist} \wedge \mathsf{Anon} \wedge \mathsf{DoSResist},\\
\mathsf{SecClaim}_{core} &\Rightarrow \mathsf{SelfCertID} \wedge \mathsf{TypedSession} \wedge \mathsf{ReplayReject},\\
\mathsf{SecClaim}_{policy} &\Rightarrow \forall z \in \mathsf{SecEvent}.\ \mathsf{Policy}_R(z),\\
\mathsf{SecClaim}_{core} &\not\Rightarrow \mathsf{SecClaim}_{policy}.
\end{aligned}$$

Obligation 4 (Replay and session-confusion rejection).

$$\begin{aligned}
\text{Accept}_\Delta(f,t) &\Rightarrow \text{SessionValid}(sid(f)) \wedge t < exp(sid(f)) \wedge \text{ctx}(f) = \text{ctx}(sid(f)) \wedge \text{fid}(f) \notin seen_{fid} \wedge seq(f) \notin seen_{seq},\\
seen'_{fid} &= seen_{fid} \cup \{\text{fid}(f)\},\\
seen'_{seq} &= seen_{seq} \cup \{seq(f)\},\\
\text{emit}(f) &\Rightarrow \text{fid}(f) \in seen'_{fid},\\
\text{DropDup}_\Delta(f) &\Rightarrow \mathsf{emit} = \epsilon.
\end{aligned}$$

X. Verification Evidence and Evaluation Scope

$$\begin{aligned}
\mathsf{Ledger} &= \mathsf{Spec} \uplus \mathsf{Artifact}_R \uplus \mathsf{ReportedOnly} \uplus \mathsf{ExtBase}_C \uplus \mathsf{Obligation},\\
\mathsf{Artifact}_R &= \mathsf{FiniteModel} \cup \mathsf{TraceReplay} \cup \mathsf{Tests}_{CI} \cup \mathsf{FrameImpl} \cup \mathsf{ChunkImpl} \cup \mathsf{VNodeImpl} \cup \mathsf{SyncImpl} \cup \mathsf{OnionReg} \cup \mathsf{OnionCircuit},\\
\mathsf{OnionImpl}_R &= \mathsf{OnionReg} \cup \mathsf{OnionCircuit},\\
\mathsf{Manifest}_R &= \mathsf{Path} \times \mathsf{Scope} \times \mathsf{Status},\\
\mathsf{AdmittedArtifact}(a) &\Leftrightarrow a \in \mathsf{Artifact}_R \wedge \mathsf{Anchor}_R(a) \in \mathsf{Manifest}_R,\\
\mathsf{PerfClaim}_R &\in \mathsf{Obligation}.
\end{aligned}$$

This section separates executable artifact evidence from reported coordinates. The repository contains finite-scope evidence for overlay, storage, onion circuit, and message-flow obligations. It does not yet contain a wide-area Rings benchmark, or a manifest entry on this paper branch with source revision, command, seed, and host metadata sufficient to regenerate the comparison tables below. Tables IV and V are the artifact ledger; reported comparison coordinates are isolated in Table VI.

$$\begin{aligned}
\mathsf{ProvFull}(d) &\Leftrightarrow \mathsf{src}(d) \neq \bot \wedge \mathsf{rev}(d) \neq \bot \wedge \mathsf{cmd}(d) \neq \bot \wedge \mathsf{seed}(d) \neq \bot \wedge \mathsf{host}(d) \neq \bot,\\
\mathsf{AdmittedEval}(d) &\Leftrightarrow \mathsf{Regen}(d) \wedge \mathsf{ProvFull}(d),\\
\neg\mathsf{AdmittedEval}(d) &\Rightarrow d \in \mathsf{ReportedOnly} \cup \mathsf{ExtBase}_C.
\end{aligned}$$

Definition 38 (Execution relation and effect abstraction). Let A_A be concrete scheduled actions and $\mathcal{I}_A$ the abstract inputs of actor A.

$$\begin{aligned}
I_A &: A_A \rightharpoonup \mathcal{I}_A,\\
I_A^\sharp(\langle a_1, \ldots, a_m\rangle) &= \langle I_A(a_j) : I_A(a_j) \downarrow\rangle_{j=1}^m.
\end{aligned}$$

For triples, lift componentwise:

$$I_A^\sharp(s,o,\beta) = (s,o,I_A^\sharp(\beta)).$$

$$\frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau_A(S,i) = (S',O,\beta)}{(S,Q) \to_A (S', Q_0 \cdot \beta)}$$

$$\begin{aligned}
\mathsf{Explore}_\Gamma &= \mathsf{ModelCheck}(\to_A, \Gamma),\\
\Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}).
\end{aligned}$$

$$\mathsf{ModelClaim}(\Gamma) \Rightarrow \mathsf{finite}(\Gamma) \wedge \mathsf{declared}(\Gamma).$$

The relation follows TLA+ style and Stateright execution in the declared finite scope [9], [10].

Definition 39 (Effect preorder).

$$(s,o,\alpha) \preceq_A (s',o',\alpha') \iff s = s' \wedge o = o' \wedge \mathsf{obs}_A(\alpha) = \mathsf{obs}_A(\alpha').$$

Here obs_A erases implementation-local stutter effects such as logging, timers already represented by Γ, and transport bookkeeping not visible in the actor interface.

Invariant 6 (Effect refinement).

$$\forall s, i. \quad I_A^\sharp(\mathrm{Impl}_A(s,i)) \preceq_A I_A^\sharp(\tau_A(s,i)).$$

Equivalently, implementation and specification have equal state/output projections and equal observable abstract effect traces:

$$\pi_{state,out}(I_A^\sharp(\mathrm{Impl}_A)) = \pi_{state,out}(I_A^\sharp \circ \tau_A).$$

Obligation 5 (Topology coverage).

$$\mathcal{L} = \{line, wrap, gap, partition, merge, churn\}$$

$$\mathcal{S} = \{loss, dup, reorder, timeout, restart\}.$$

$$\mathsf{CheckCase} = (L, S, P), \qquad L \in \mathcal{L},\ S \in \mathcal{S}.$$

Artifact	Checked scope	Non-claim / limit
Topology tests	join, rectify, stabilize fixpoints	no parametric TLAPS proof
Stateright	predecessor, discovery, CRDT, handoff	finite Γ, no live-network theorem
Trace replay	deterministic routing schedules	no Internet latency benchmark
E2E tests	signed frames, ordering, replay	no symbolic crypto proof
Storage VNODE	owner derivation, route, sync, repair	no full routing-VNODE or perf theorem
Onion registry	signed exits, live directory, route selection	no hidden-service or anonymity theorem
Onion circuit	layered AEAD, relay/exit replay tests	no traffic-analysis or DoS theorem
CI witness	build, test, lint, doc, package	no benchmark regeneration

Table IV
Verification evidence and explicit limits. Entries are finite-scope implementation evidence; none is a wide-area Rings lookup benchmark.

Symbol	Artifact anchor	Admission status
FiniteModel	dht_stateright.rs	admitted only for finite Γ
TraceReplay	trace_replay.rs	deterministic schedules
Tests_{CI}	qaci.yml	build/test witness, no benchmark regen
FrameImpl	message/e2e.rs	signed stream-frame behavior
ChunkImpl	chunk.rs	bounded whole-message reassembly
VNodeImpl	virtual_node.rs	storage owner derivation
OnionReg	onion/mod.rs, route.rs	signed exit directory, route builder
OnionCircuit	circuit/*.rs	layered AEAD relay/exit behavior
SyncImpl	storage/*.rs	additive repair, guarded cleanup

Table V
Artifact manifest for the current paper claim ledger. Each row names the repository anchor that can support an admitted finite-scope claim.

Definition 40 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness}.$$

$$h = O(\log |N|) \quad (\mathrm{Inv}_{ring}(N,S) \wedge \mathsf{FingerInv}(N,S)).$$

Only hT_{hop} is Chord-comparable; the rest is Rings runtime overhead.

Symbol	Coordinate source	Ledger status
D_C^{paper}	Chord paper table	$\mathsf{ExtBase}_C$, external baseline
D_C^{656}	PR 656 Chord-only dummy benchmark	$\mathsf{ReportedOnly} \cap \mathsf{Baseline}_C$
D_R^{docker}	reported Docker/WebRTC run	$\mathsf{ReportedOnly} \cap \mathsf{Runtime}_R$
D_R^{svnode}	PR 656 storage-VNODE load rows	$\mathsf{ReportedOnly} \cap \mathsf{Runtime}_R$

Table VI
Reported coordinate ledger. These rows calibrate comparison or runtime context; they are not admitted artifact evidence until provenance is complete.

Definition 41 (Benchmark data model).

$$\begin{aligned}
\eta &= (N, r, m, L, f, \lambda, \gamma, \mu_B),\\
\mu_B &\in \{\mathsf{maintained}, \mathsf{stale}\},\\
\pi(d) &\in \mathsf{Prov},\\
\mathsf{Regen}(d) &\Rightarrow \mathsf{src}(d) \neq \bot \wedge \mathsf{rev}(d) \neq \bot\\
&\quad \wedge \mathsf{cmd}(d) \neq \bot \wedge \mathsf{seed}(d) \neq \bot\\
&\quad \wedge \mathsf{host}(d) \neq \bot.
\end{aligned}$$

$$\begin{aligned}
F_C(\eta; D) &\subseteq Q_C(\eta; D),\\
\phi_C(\eta; D) &= |F_C(\eta; D)|/|Q_C(\eta; D)|,\\
ok_C(\eta; D) &= 1 - \phi_C(\eta; D),\\
\bar{to}_C(\eta; D) &= |Q_C|^{-1} \textstyle\sum_{q \in Q_C} to_C(q),\\
B_C(\eta; D) &= (\mathbb{E}[h_C^f \mid \eta, D], \mathbb{E}[h_C^r \mid \eta, D], \bar{to}_C, ok_C, \phi_C),\\
h_C^r &= h_C^f + 1.
\end{aligned}$$

$$\begin{aligned}
D_{external} &= D_C^{paper},\\
D_{reported} &= D_C^{656} \uplus D_R^{docker} \uplus D_R^{svnode},\\
D_{all} &= D_{external} \uplus D_{reported},\\
D_{admit} &= \{d \in D_{all} : \mathsf{AdmittedEval}(d)\},\\
\mathcal{F}_{II} &= \{0, .10, .20, .30, .40, .50\},\\
\Lambda_{III} &= \{.05, .10, .15, .20, .25, .30, .35, .40\},\\
D_C^{paper} &= \{C_{II}(f) : f \in \mathcal{F}_{II}\}\\
&\quad \uplus \{C_{III}(\lambda) : \lambda \in \Lambda_{III}\},\\
D_{maint}^{656} &= \{stable, f_{.10}, f_{.20}, \gamma_{.05}, \gamma_{.40}\},\\
D_{stale}^{656} &= \{f_{.10}, f_{.20}, \gamma_{.05}, \gamma_{.40}\},\\
D_C^{656} &= (D_{maint}^{656} \times \{\mathsf{maintained}\})\\
&\quad \uplus (D_{stale}^{656} \times \{\mathsf{stale}\}),\\
D_R^{docker} &= \{D_R^{conv}, D_R^{tx}\},\\
D_R^{svnode} &= \{\ell_v : v \in \{1, 2, 5, 10, 20\}\}.
\end{aligned}$$

$$\begin{aligned}
D_C^{paper} &: (N, r) = (1000, 20),\\
D_C^{656} &: (N, r, m, L) = (1600, 3, 160, 64),\\
D_C^{paper} &\subset \mathsf{ExtBase}_C,\\
D_C^{656} &\subset \mathsf{ReportedOnly} \cap \mathsf{Baseline}_C,\\
D_R^{docker} &\subset \mathsf{ReportedOnly} \cap \mathsf{Runtime}_R,\\
D_R^{svnode} &\subset \mathsf{ReportedOnly} \cap \mathsf{Runtime}_R,\\
\mathsf{Baseline}_C \cap \mathsf{Runtime}_R &= \varnothing.
\end{aligned}$$

Here D_C^{paper} is an external Chord-paper coordinate, D_C^{656} is a PR 656 Chord-only coordinate at current Rings overlay parameters, and D_R^{docker} and D_R^{svnode} are reported Rings runtime evidence. The sets are disjoint by claim role: external baseline, reported Chord baseline, reported Rings runtime. At this paper revision, PR 656 coordinates are not artifact-ledger rows because the paper branch records the numbers but does not admit a manifest row $(d, \mathsf{Ev}_\Gamma, a, \sigma)$ with regeneration metadata. Unless

$\mathsf{AdmittedEval}(d)$ holds, a tuple is a reported coordinate only; it is not used to prove Core_{Rings} or any Rings performance claim. Metrics are not pooled unless a Rings lookup benchmark with the same workload key η is added.

Table VII records the Chord-paper baseline coordinates ($N = 1000, r = 20$). It calibrates comparison scale only; it is neither Rings runtime evidence nor generated by this repository.

η	N, r	$\mathbb{E}[h_C^f]$	timeout_C	$10^4\,\text{fail}_C$
$C_{II}, f = 0$	$1000, 20$	3.815	0.000	0.0
$C_{II}, f = .10$	$1000, 20$	4.007	0.522	0.0
$C_{II}, f = .20$	$1000, 20$	4.177	1.071	0.0
$C_{II}, f = .30$	$1000, 20$	4.354	1.830	0.0
$C_{II}, f = .40$	$1000, 20$	4.791	3.700	0.0
$C_{II}, f = .50$	$1000, 20$	5.345	6.565	0.0
$C_{III}, \lambda = .05$	$\simeq 1000, 20$	3.905	0.076	7.0
$C_{III}, \lambda = .10$	$\simeq 1000, 20$	3.949	0.168	19.0
$C_{III}, \lambda = .15$	$\simeq 1000, 20$	4.044	0.272	18.0
$C_{III}, \lambda = .20$	$\simeq 1000, 20$	4.094	0.335	35.0
$C_{III}, \lambda = .25$	$\simeq 1000, 20$	4.128	0.405	36.0
$C_{III}, \lambda = .30$	$\simeq 1000, 20$	4.221	0.501	47.0
$C_{III}, \lambda = .35$	$\simeq 1000, 20$	4.308	0.599	60.0
$C_{III}, \lambda = .40$	$\simeq 1000, 20$	4.345	0.656	58.0

Table VII

External Chord-paper baseline coordinates ($N = 1000, r = 20$), transcribed as reported comparison coordinates. They are not Rings runtime evidence and are not regenerated by the current artifact.

Table VIII fixes a Chord-only baseline at current Rings overlay parameters ($N = 1600, r = 3, m = 160, L = 64$). Maintained rows are the comparison baseline; stale rows intentionally disable maintenance and measure sensitivity, not Rings behavior.

η	model	$\bar{h}^f/\bar{h}^r$	ok	$\bar{to}$	$10^4\phi$
stable	maint.	4.469/5.469	1.000	0.000	0.0
$f = .10$	maint.	4.394/5.394	1.000	0.000	0.0
$f = .20$	maint.	4.316/5.316	1.000	0.000	0.0
$\gamma = .05$	maint.	4.465/5.465	1.000	0.000	0.0
$\gamma = .40$	maint.	4.470/5.470	1.000	0.000	0.0
$f = .10$	stale	4.758/5.758	.990	.734	98.2
$f = .20$	stale	5.186/6.186	.969	1.890	314.7
$\gamma = .05$	stale	4.617/5.617	.945	.361	546.2
$\gamma = .40$	stale	5.746/6.746	.484	4.604	5161.1

Table VIII

Chord-only baselines at Rings parameters ($N = 1600, r = 3, m = 160, L = 64$). Maintained rows are the baseline; stale rows are no-maintenance sensitivity. These are PR 656 reported coordinates, not admitted current-artifact evidence in this paper branch until generator, revision, seed, command, and host enter $\mathsf{Manifest}_R$.

Thus stale rows explain sensitivity, not the maintained Chord baseline:

$$\begin{aligned} \mu_B = \mathsf{stale} &\Rightarrow \neg\mathsf{maintenance}, \\ \mu_B = \mathsf{maintained} &\Rightarrow \mathsf{Baseline}_C^{656}. \end{aligned}$$

Table IX records the PR 656 storage-VNODE supplement after PR 660. It uses the runtime StorageVirtualNodes API for storage-owner selection over 160-bit Rings DIDs:

$$\begin{aligned} N_R, K, m, net &= (10^4, 10^6, 160, 1619), \\ \ell_v &= (N_R, K, m, net, v), \\ \mathsf{scope}(\ell_v) &= \mathsf{storage\_owner\_selection}, \\ \mathsf{routingVNode}(\ell_v) &= \mathsf{false}. \end{aligned}$$

v	$\lvert V\rvert$	mean	p1	p99	max
1	10000	100.0	0	457	1164
2	20000	100.0	6	327	695
5	50000	100.0	23	236	392
10	100000	100.0	38	194	263
20	200000	100.0	52	165	228

Table IX

Reported storage-VNODE load-balance coordinates from PR 656 (chord-paper-sim-2026-07-08.jsonl, commit aa89fc4). Rows use PR 660 runtime storage virtual-node derivation. They are storage owner-placement data, not routing/finger VNODE data and not WebRTC execution.

Obligation 6 (Benchmark report).

$$\begin{aligned} \mathsf{Report}_{eval} = (&\mathsf{Baseline}_C^{paper}, \mathsf{Baseline}_C^{656}, \\ &\mathsf{Runtime}_R^{docker}, \mathsf{Runtime}_R^{svnode}, \\ &\mathsf{Prov}, \mathsf{Limit}). \end{aligned}$$

$$\begin{aligned} \mathsf{ReportCmp}_{lookup} &= B_C(\eta; D_C^{paper} \uplus D_C^{656}), \\ \mathsf{Measure}_{runtime} &= B_R^{docker}(\eta_R; D_R^{docker}), \\ \mathsf{Measure}_{svnode} &= B_R^{svnode}(\ell; D_R^{svnode}), \\ \mathsf{ReportCmp}_{lookup} &\not\equiv \mathsf{Measure}_{runtime}, \\ \mathsf{ReportCmp}_{lookup} &\not\equiv \mathsf{Measure}_{svnode}. \end{aligned}$$

A Rings performance claim must measure Rings under the reported build; a Chord baseline may only be cited as a baseline.

$$\begin{aligned} \mathsf{AdmitPerf}_R(\eta) \;\Leftrightarrow\; & \exists D_R^{lookup}. \\ & \mathsf{AdmittedEval}(D_R^{lookup}) \\ & \wedge\, workload(D_R^{lookup}) = \eta \\ & \wedge\, metric(D_R^{lookup}) \supseteq \{h, T_{hop}, T_{conn}, \\ & T_{crypto}, T_{app}, T_{witness}\}, \\ \neg\mathsf{AdmitPerf}_R(\eta) \;\Rightarrow\; & \mathsf{PerfClaim}_R(\eta) \\ & \in \mathsf{Obligation}. \end{aligned}$$

$$\begin{aligned} \vec{T}_{total} &= (T_{conn}, hT_{hop}, T_{crypto}, \\ &\quad T_{app}, T_{witness}), \\ \mathsf{Ccmp}(\vec{T}_{total}) &= hT_{hop}, \\ \mathsf{Rovh} &= (T_{conn}, T_{crypto}, T_{app}, \\ &\quad T_{witness}, \mathsf{runtime}). \end{aligned}$$

Only hT_{hop} is Chord-comparable. Connection establishment, cryptography, application handling, witness generation, and runtime scheduling are Rings overheads and require Rings measurements.

$$\begin{aligned} D_R^{conv} &= \mathsf{conv}(n = 16, T_{wall} = 309.532\,\mathrm{s}, \\ &\quad fail = 0/16, partial = 3/16, \\ &\quad t_{p50} = .194, t_{p95} = 1.026, \mathsf{msg}_s = 8058.6), \\ D_R^{tx} &= \{\mathsf{tx}(1, 10.46), \mathsf{tx}(16, 107.35), \\ &\quad \mathsf{tx}(64, 90.36)\}. \end{aligned}$$

$$\mathsf{tx} : \mathrm{KiB} \to \mathrm{Mbps}, \qquad \mathsf{msg}_s : \mathrm{messages/s}.$$

The Docker/WebRTC tuple is convergence evidence, not a steady-state lookup benchmark. D_R^{tx} is payload throughput; it must not be compared to Chord forwarding hops or timeout rates.

Artifact provenance.

$$\begin{aligned}\mathsf{ProvFull}(d) &\Leftrightarrow \mathsf{src}(d) \neq \bot \wedge \mathsf{rev}(d) \neq \bot \\ &\quad \wedge \mathsf{cmd}(d) \neq \bot \wedge \mathsf{seed}(d) \neq \bot \\ &\quad \wedge \mathsf{host}(d) \neq \bot, \\ \mathsf{AdmittedEval}(d) &\Leftrightarrow \mathsf{Regen}(d) \wedge \mathsf{ProvFull}(d).\end{aligned}$$

No CI command currently regenerates Table VII, Table VIII, Table IX, D_R^{conv}, or D_R^{tx}. Therefore PR 656 maintained-Chord and storage-VNODE rows are useful reported coordinates, but they are not admitted evidence for a Rings performance theorem in this revision.

Limitations. The current artifact supports finite model/test evidence, storage-VNODE implementation evidence, onion exit/circuit implementation evidence, reported storage-VNODE load-balance coordinates, and reported baseline coordinates. It does not yet support claims about wide-area latency, NAT population behavior, adversarial churn beyond the scoped models, symbolic cryptographic security, full routing VNODEs, storage-VNODE runtime performance, onion anonymity or traffic-analysis resistance, exit-abuse resistance, or steady-state Rings lookup performance.

XI. Related Work

Self-certifying systems bind names to keys, while DID standardization separates identifier syntax, verification methods, and resolution metadata [11], [12]. The Rings design follows that line: DID control is a cryptographic predicate, while resource placement is $\Omega_N(X, H) = (X, \mathrm{owner}_N \circ H)$, not a global certificate authority or ledger.

Protection systems and authorization logics make authority an explicit predicate; monads and algebraic effects make runtime effects typed [13]–[16]. This specification uses them as a composition discipline: authority, placement, and effect emission live in separate carriers and meet only through typed morphism laws.

Structured overlays give different placement laws. Chord gives clockwise successor ownership; Kademlia/IPFS/libp2p give XOR or provider-record lookup; Pastry, Tapestry, and CAN explore prefix, proximity, and coordinate-space routing [4], [17]–[22]. The claimed contribution is not another lookup metric; it is the separation between the circular owner law, DID/session authority, CRDT-style storage merge, and browser/native transport interpretation.

Security work on DHTs shows why this separation must be explicit. Secure routing, Sybil, and Eclipse attacks require admission, identifier assignment, neighbor-set constraints, auditing, or redundant routing beyond ordinary DHT maintenance [23]–[26]. This paper therefore states Sybil, Eclipse, anonymity, and DoS resistance as policy obligations, not as consequences of CorrectChord.

Anonymity systems make traffic analysis a primary security objective. Tor uses onion routing; Nym adds mix-network style privacy infrastructure [27], [28]. This paper does not prove sender, receiver, or route unlinkability. The Rings relay layer is only a typed transport policy:

$$\mathsf{P}_{relay} \not\Rightarrow \mathsf{Anon}.$$

The implementation nevertheless now includes an application-layer onion exit registry and route-aware circuit data plane:

$$\begin{aligned}\mathsf{OnionImpl}_R &= \mathsf{OnionReg} \cup \mathsf{OnionCircuit}, \\ \mathsf{OnionReg} &= \mathsf{ExitRegistry} \cup \mathsf{RouteBuilder}, \\ \mathsf{OnionCircuit} &= \mathsf{LayeredAEAD}, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{Anon}, \\ \mathsf{OnionImpl}_R &\not\Rightarrow \mathsf{Unlinkability}.\end{aligned}$$

The Rings storage layer uses the join-semilattice CRDT case when merge is available and an explicit finalizer otherwise [6]. WebRTC and WebAssembly are runtime carriers, not overlay algorithms; WebRTC measurement work also suggests that connection setup, NAT traversal, and payload throughput must be reported separately from DHT hop count [29]–[31].

XII. Conclusion

This paper should be read as a bounded theorem plus a claim ledger for the Rings implementation, not as an informal bundle of peer-to-peer mechanisms:

$$\mathsf{Model}_{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned}\mathsf{Formalized}_{paper} &= \mathsf{Core}_{Rings}, \\ \mathsf{LedgerRule}_{paper} &= \mathsf{LedgerDisc}_{Rings}, \\ \mathsf{ScopedEv} &= \{a : a \in \mathsf{Artifact}_R \\ &\quad \wedge \mathsf{AdmittedArtifact}(a)\}, \\ \mathsf{RepCoord} &= D_C^{paper} \uplus D_C^{656} \uplus D_R^{docker} \\ &\quad \uplus D_R^{svnode}, \\ \mathsf{Obligation} &= \mathsf{wideAreaBenchmark} \\ &\quad \cup \mathsf{adversarialLiveness} \\ &\quad \cup \mathsf{fullReplicaRealization} \\ &\quad \cup \mathsf{AdmittedStorageVNodeLoad}, \\ \mathsf{NonClaim} &= \mathsf{NonClaim}_{Rings}.\end{aligned}$$

The scientific contribution is the composition law and its claim ledger: CorrectChord supplies the overlay safety root, storage VNODEs instantiate the default $\mathrm{StoreOwner}_N^{v_0}$ placement map, Ω_N lifts resource namespaces into live owners, onion circuits instantiate a typed application data plane, protocol morphisms preserve placement, and runtime effects remain typed. Virtual positions are storage coordinates, not authority principals or transport endpoints. Onion routes are encrypted circuit coordinates, not anonymity proofs. $\mathsf{Formalized}_{paper}$ is a theorem inside the stated model; $\mathsf{LedgerRule}_{paper}$ is the admission rule for paper claims. Implementation evidence supports only the entries admitted by Table V; coordinates in Table VI are not proof evidence. Benchmark and security claims outside that ledger remain obligations.

References

[1] N. Kshetri, "Privacy and security issues in cloud computing: The role of institutions and institutional evolution," Telecommunications Policy, vol. 37, no. 4–5, pp. 372–386, 2013.

[2] D. Zissis and D. Lekkas, "Addressing cloud computing security issues," Future Generation Computer Systems, vol. 28, no. 3, pp. 583–592, 2012.

[3] N. Kshetri and S. Murugesan, "Cloud computing and EU data privacy regulations," Computer, vol. 46, no. 3, pp. 86–89, 2013.

[4] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications. ACM, 2001, pp. 149–160.

[5] P. Zave, "Reasoning about identifier spaces: How to make Chord correct," IEEE Transactions on Software Engineering, vol. 43, no. 12, pp. 1144–1156, 2017, https://arxiv.org/abs/1610.01140.

[6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[7] A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[8] B. Campbell, R. Mahy, and C. Jennings, "The message session relay protocol (MSRP)," RFC 4975, 2007, https://www.rfc-editor.org/info/rfc4975.

[9] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[10] J. Nadal, "Stateright: Model checking for implementing distributed systems," https://www.stateright.rs/, accessed: July 4, 2026.

[11] D. Mazières, M. Kaminsky, M. F. Kaashoek, and E. Witchel, "Separating key management from file system security," in Proceedings of the 17th ACM Symposium on Operating Systems Principles, 1999, pp. 124–139.

[12] W3C Decentralized Identifier Working Group, "Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations," W3C Recommendation, 2022, https://www.w3.org/TR/2022/REC-did-core-20220719/.

[13] B. W. Lampson, "Protection," ACM SIGOPS Operating Systems Review, vol. 8, no. 1, pp. 18–24, 1974.

[14] M. Abadi, M. Burrows, B. Lampson, and G. Plotkin, "A calculus for access control in distributed systems," ACM Transactions on Programming Languages and Systems, vol. 15, no. 4, pp. 706–734, 1993.

[15] E. Moggi, "Notions of computation and monads," Information and Computation, vol. 93, no. 1, pp. 55–92, 1991.

[16] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in Programming Languages and Systems, ser. Lecture Notes in Computer Science, vol. 5502, 2009, pp. 80–94.

[17] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.

[18] J. Benet, "IPFS - content addressed, versioned, P2P file system," arXiv:1407.3561 [cs.NI], 2014, https://arxiv.org/abs/1407.3561.

[19] Protocol Labs, "libp2p: A modular network stack," 2026, https://libp2p.io/, accessed 2026-07-07.

[20] A. Rowstron and P. Druschel, "Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems," in Middleware 2001, ser. Lecture Notes in Computer Science, vol. 2218, 2001, pp. 329–350.

[21] B. Y. Zhao, L. Huang, J. Stribling, S. C. Rhea, A. D. Joseph, and J. Kubiatowicz, "Tapestry: A resilient global-scale overlay for service deployment," IEEE Journal on Selected Areas in Communications, vol. 22, no. 1, pp. 41–53, 2004.

[22] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A scalable content-addressable network," in Proceedings of ACM SIGCOMM, 2001, pp. 161–172.

[23] E. Sit and R. Morris, "Security considerations for peer-to-peer distributed hash tables," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429, 2002, pp. 261–269.

[24] M. Castro, P. Druschel, A. Ganesh, A. Rowstron, and D. S. Wallach, "Secure routing for structured peer-to-peer overlay networks," in 5th USENIX Symposium on Operating Systems Design and Implementation, 2002.

[25] J. R. Douceur, "The sybil attack," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429, 2002, pp. 251–260.

[26] A. Singh, T.-W. J. Ngan, P. Druschel, and D. S. Wallach, "Eclipse attacks on overlay networks: Threats and defenses," in 25th IEEE International Conference on Computer Communications, 2006, pp. 1–12.

[27] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The Second-Generation onion router," in 13th USENIX Security Symposium (USENIX Security 04). San Diego, CA: USENIX Association, Aug. 2004, https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router.

[28] C. Diaz, H. Halpin, and A. Kiayias, "The Nym network: The next generation of privacy infrastructure," Whitepaper, 2021, https://nym.com/nym-whitepaper.pdf.

[29] H. Alvestrand, "Overview: Real-time protocols for browser-based applications," RFC 8825, 2021, https://www.rfc-editor.org/rfc/rfc8825.

[30] WebAssembly Working Group, "WebAssembly core specification," W3C Recommendation, 2019, https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/.

[31] K. Nakagawa, M. Tsukada, K. Shima, and H. Esaki, "WebRTC-based measurement tool for peer-to-peer applications and preliminary findings with real users," arXiv:2112.02163, 2021, https://arxiv.org/abs/2112.02163.